



Software Development Methodology

A **software development methodology** is a structured approach or framework that guides how software projects are planned, executed, and delivered. It defines the **processes, practices, and rules** teams follow to ensure software is built efficiently, meets user needs, and aligns with business goals.

Different methodologies exist to suit different project contexts — some emphasize **predictability and documentation** (like Waterfall), while others focus on **flexibility and collaboration** (like Agile and Scrum).



Waterfall and Agile Methodology

Waterfall is a traditional, linear software development methodology where each phase must be completed before moving to the next, while Agile is an iterative, flexible approach that emphasizes collaboration, adaptability, and continuous delivery.

Waterfall works best for projects with fixed requirements, whereas Agile thrives in dynamic environments where change is expected.

- Use **Waterfall** when requirements are fixed, risks are predictable, and documentation is critical.
- Use **Agile** when requirements evolve, customer feedback is essential, and speed of delivery matters.



Waterfall Methodology

A linear, sequential approach to software development where each phase (**requirements** → **design** → **development** → **testing** → **deployment** → **maintenance**) must be completed before moving to the next. Best suited for projects with stable, well-defined requirements.

Approach: Sequential and phase-driven — **requirements** → **design** → **development** → **testing** → **deployment** → **maintenance**.

Strengths:

- Clear structure and documentation.
- Predictable timelines and deliverables.
- Easier to manage in regulated industries.

Limitations:



- Inflexible to change once requirements are set.
- Feedback comes late (after testing).
- Risk of misalignment if requirements evolve.

Best For: Projects with **well-defined, stable requirements** (e.g., government systems, compliance-heavy applications).

Example: An **online food delivery system** built with Waterfall would fully document requirements (menus, payment, delivery tracking) before design, then code all features sequentially, test at the end, and deploy only after all phases are complete.



Agile Methodology

An iterative and adaptive approach that delivers work in short cycles (sprints). It emphasizes collaboration, customer feedback, and flexibility, allowing teams to respond quickly to changing requirements.

Approach: Iterative and adaptive — work delivered in short cycles (sprints or iterations).

Core Values (Agile Manifesto):

- **Individuals & interactions** over processes/tools.
- **Working software** over documentation.
- **Customer collaboration** over contract negotiation.
- **Responding to change** over following a rigid plan.

Strengths:

- Flexibility to adapt to evolving requirements.
- Continuous feedback and faster delivery of value.
- Strong collaboration across cross-functional teams.

Limitations:

- Requires disciplined, self-organizing teams.
- Less emphasis on formal documentation.
- Can be challenging for large, distributed teams.



Best For: Projects in **dynamic environments** (e.g., startups, product innovation, customer-centric apps).

Example: An **HR management system** built with Agile would deliver features incrementally — first employee registration, then leave management, then payroll — with user feedback shaping each sprint.



Comparison



<u>Aspect</u>	<u>Waterfall</u>	<u>Agile</u>
Process	Linear, sequential	Iterative, adaptive
Flexibility	Low — hard to change	High — embraces change
Documentation	Heavy, detailed	Lightweight, minimal
Feedback	Late (after testing)	Continuous (every sprint)
Best For	Stable requirements, compliance projects	Dynamic requirements, innovation projects

- Use **Waterfall** when requirements are fixed, risks are predictable, and documentation is critical.
- Use **Agile** when requirements evolve, customer feedback is essential, and speed of delivery matters.



Scrum

A **framework within Agile** that organizes work into sprints (time-boxed iterations). It defines roles, events, and artifacts to help teams deliver value incrementally.

Scrum Framework

A **specific Agile framework** that organizes work into sprints (2–4 weeks).

Key Roles: Product Owner, Scrum Master, Development Team.

Artifacts: Product Backlog, Sprint Backlog, Increment.

Events: Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective.



Scrum Team

A **cross-functional group** responsible for delivering increments of value.

It includes:

Product Owner (PO): Owns the product vision and backlog (defines priorities).

Scrum Master: Facilitates the Scrum process and removes blockers (facilitates process).

Development Team: Builds and delivers the product increment (build the product).



Sprints

Short, fixed-length iterations (usually 2–4 weeks) during which the Scrum Team delivers a **potentially shippable product increment**.



Sprint Backlog

A **subset of the product backlog** chosen for a sprint. It contains tasks and user stories the team commits to completing within that sprint.



Backlog

A **prioritized list of work items** (features, enhancements, bug fixes, technical tasks) maintained by the Product Owner. It evolves continuously based on feedback and business needs.



Task

A **small, actionable unit of work** derived from a user story or backlog item. Tasks are assigned to team members and tracked during the sprint.



Micro

Refers to **small-scale, detailed activities** or granular tasks within a project. Example: writing a login function in a system.



Macro

Refers to **large-scale, high-level activities** or overall project goals. Example: delivering the entire HR management system as a product.

✨ Insight —

- **Waterfall** = rigid, sequential.
- **Agile/Scrum** = flexible, iterative.
- **Backlogs & sprints** = tools for prioritization and delivery.
- **Micro vs Macro** = zooming in on tasks vs zooming out on overall goals.



Methodology Comparison



<u>Aspect</u>	<u>Waterfall</u>	<u>Agile</u>	<u>Scrum</u>
Approach	Linear, sequential phases	Iterative, adaptive cycles	Agile framework with defined roles/events
Flexibility	Low — changes are costly	High — embraces change	High — adapts each sprint
Process Flow	Requirements → Design → Development → Testing → Deployment → Maintenance	Continuous iterations with feedback	Work divided into 2–4 week sprints
Documentation	Heavy, detailed	Lightweight, minimal	Moderate — backlog & sprint artifacts
Feedback	Late (after testing)	Continuous (every iteration)	Continuous (daily stand-ups, sprint reviews)
Team Structure	Hierarchical, phase-based	Cross-functional, collaborative	Defined roles: Product Owner, Scrum Master, Development Team
Best For	Stable requirements, compliance projects	Dynamic requirements, innovation projects	Projects needing structured Agile delivery
Limitations	Inflexible, risk of misalignment	Requires disciplined teams, less formal docs	Can be challenging for large/distributed teams

✨ Insight ✨

- **Waterfall** ensures predictability and structure.
- **Agile** drives adaptability and customer collaboration.
- **Scrum** provides a practical Agile framework with sprints, backlogs, and defined roles.